

AI Assistant Coding

Assignment-1

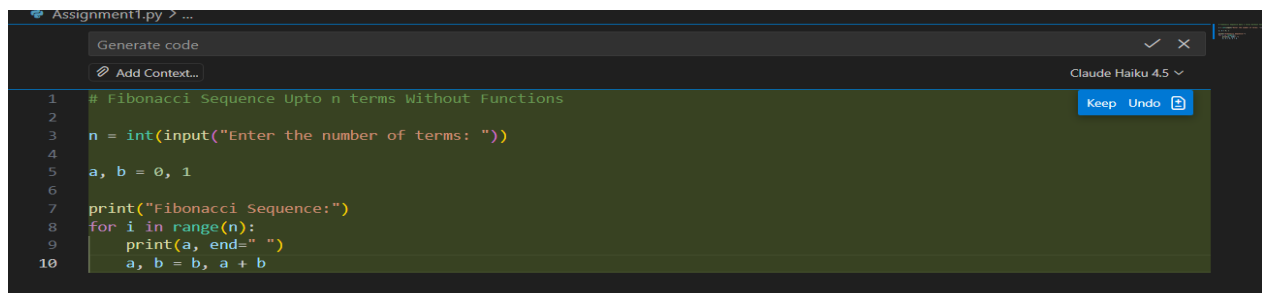
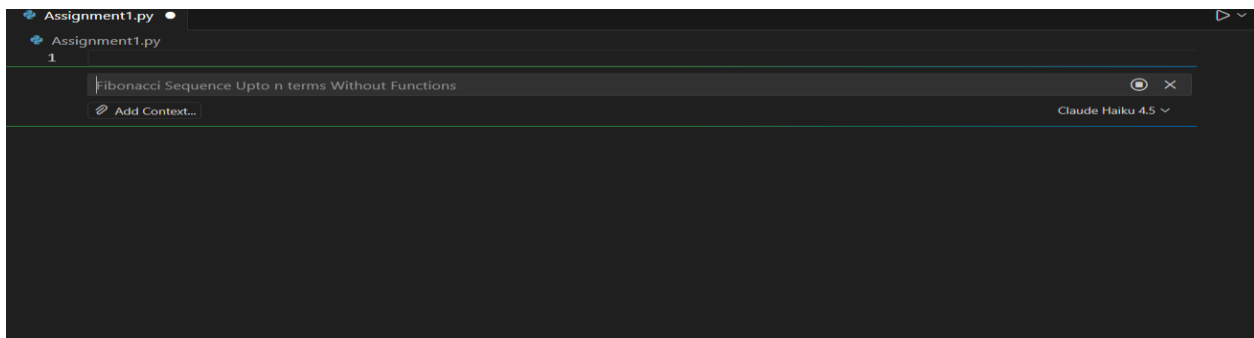
Name: M.Nandhitha

HT_NO.:2303A51352

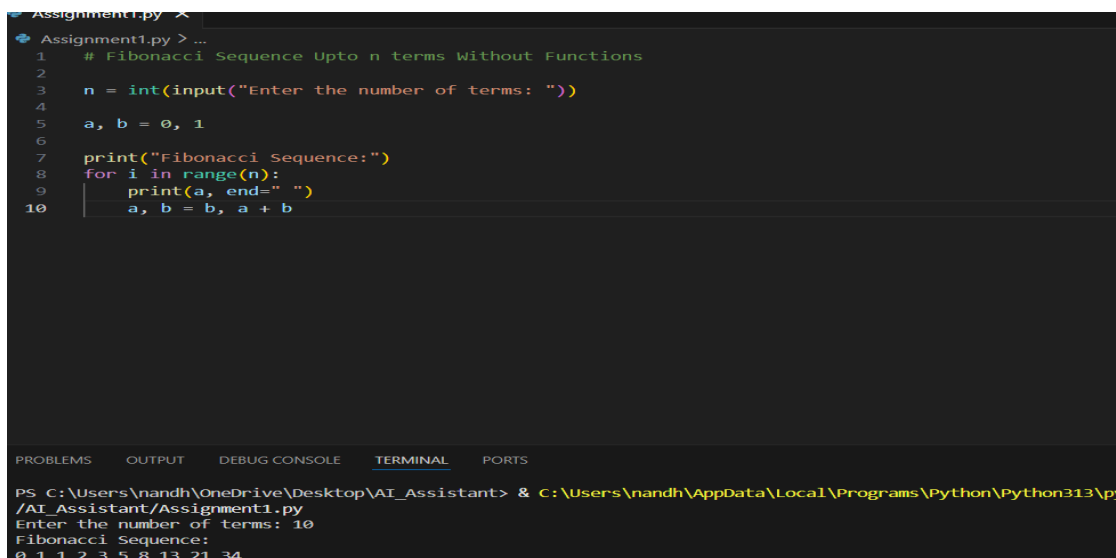
Batch:20

Task 1: AI-Generated Logic Without Modularization (Fibonacci Sequence Without Functions)

Prompt: Fibonacci Sequence upto n terms Without Functions



Result:

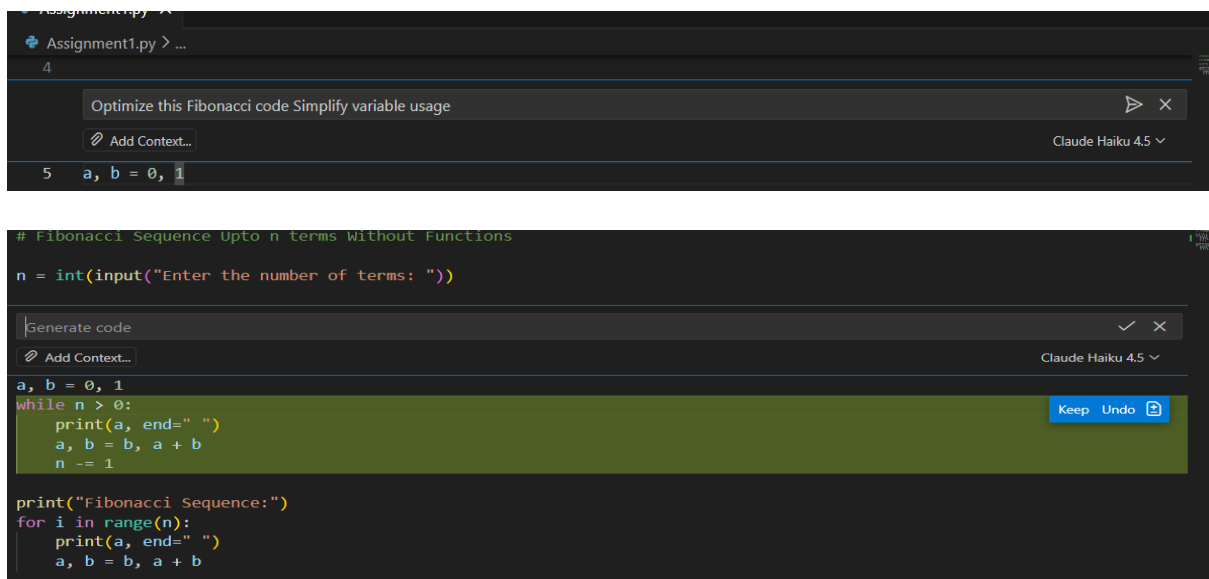


Explanation:

GitHub Copilot was able to generate a correct Fibonacci sequence using direct logic without user-defined functions. The code was simple and suitable for small, one-time scripts. However, the logic and input/output were tightly coupled, which reduced readability and reusability. This task demonstrated Copilot's ability to quickly produce working code but also highlighted limitations in scalability.

Task 2: AI Code Optimization & Cleanup (Improving Efficiency)

Prompt: Optimize this Fibonacci code Simplify variable usage



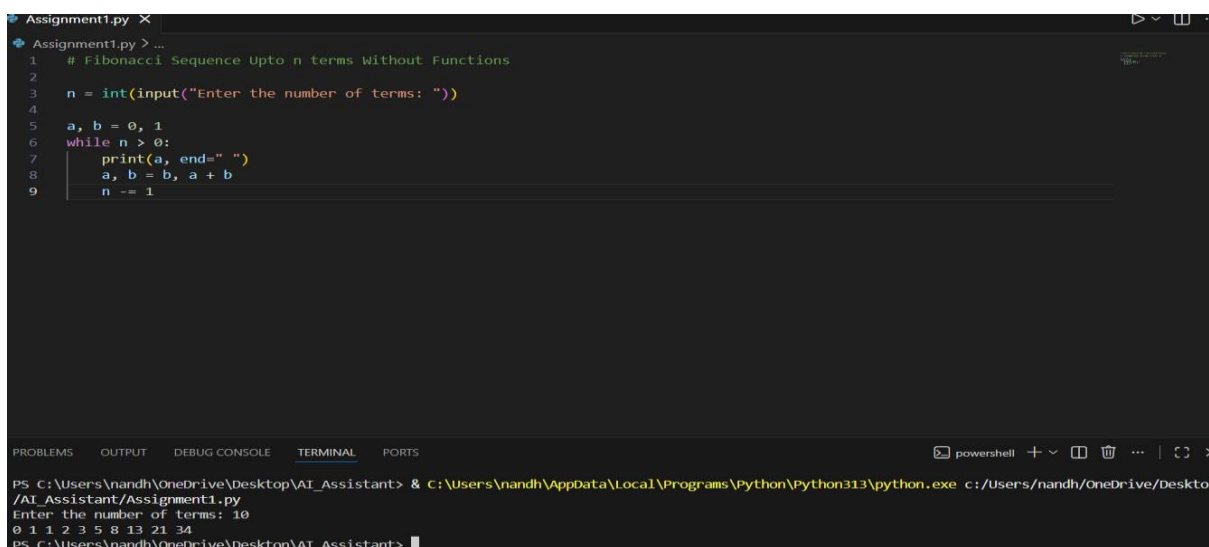
The screenshot shows a VS Code editor with a file named 'Assignment1.py'. A Copilot chat window is open with the prompt 'Optimize this Fibonacci code Simplify variable usage'. The generated code is as follows:

```
# Fibonacci Sequence Upto n terms Without Functions
n = int(input("Enter the number of terms: "))

a, b = 0, 1
while n > 0:
    print(a, end=" ")
    a, b = b, a + b
    n -= 1

print("Fibonacci Sequence:")
for i in range(n):
    print(a, end=" ")
    a, b = b, a + b
```

Result:



The screenshot shows the final code in 'Assignment1.py' and its execution output in the terminal. The code is identical to the one generated by Copilot. The terminal output shows the program running and printing the Fibonacci sequence for 10 terms.

```
PS C:\Users\nandh\OneDrive\Desktop\AI_Assistant> & C:\Users\nandh\AppData\Local\Programs\Python\Python313\python.exe c:/Users/nandh/OneDrive/Desktop/AI_Assistant/Assignment1.py
Enter the number of terms: 10
0 1 1 2 3 5 8 13 21 34
PS C:\Users\nandh\OneDrive\Desktop\AI_Assistant>
```

Explanation:

1.Original Code:

```
n=int(input("Enter the number of terms:"))  
  
a,b=0,1  
  
print("Fibonacci Sequence:")  
  
for i in range(n):  
    print(a,end=" ")  
  
    a,b=b,a+b
```

2.Optimised Code:

```
n=int(input("Enter the number of terms:"))  
  
a,b=0,1  
  
while n>0:  
    print(a,end=" ")  
  
    a,b=b,a+b  
  
    n-=1
```

3.What was inefficient:

- The loop variable i in the for loop was not used anywhere in the logic.
- Extra print statement reduced code compactness.
- Loop control was less direct for a simple decrement-based process.

4.How the optimized version improves performance and readability:

- Removed the unused loop variable, reducing redundancy.
- Used a while loop with a clear termination condition.
- Reduced code size and made the logic easier to understand.
- Maintains the same time complexity while improving clarity and cleanliness.

Task 3: Modular Design Using AI Assistance (Fibonacci Using Functions)

Prompt: Fibonacci Sequence upto n terms using functions

Generate code

✓ X

Add Context...

Claude Haiku 4.5

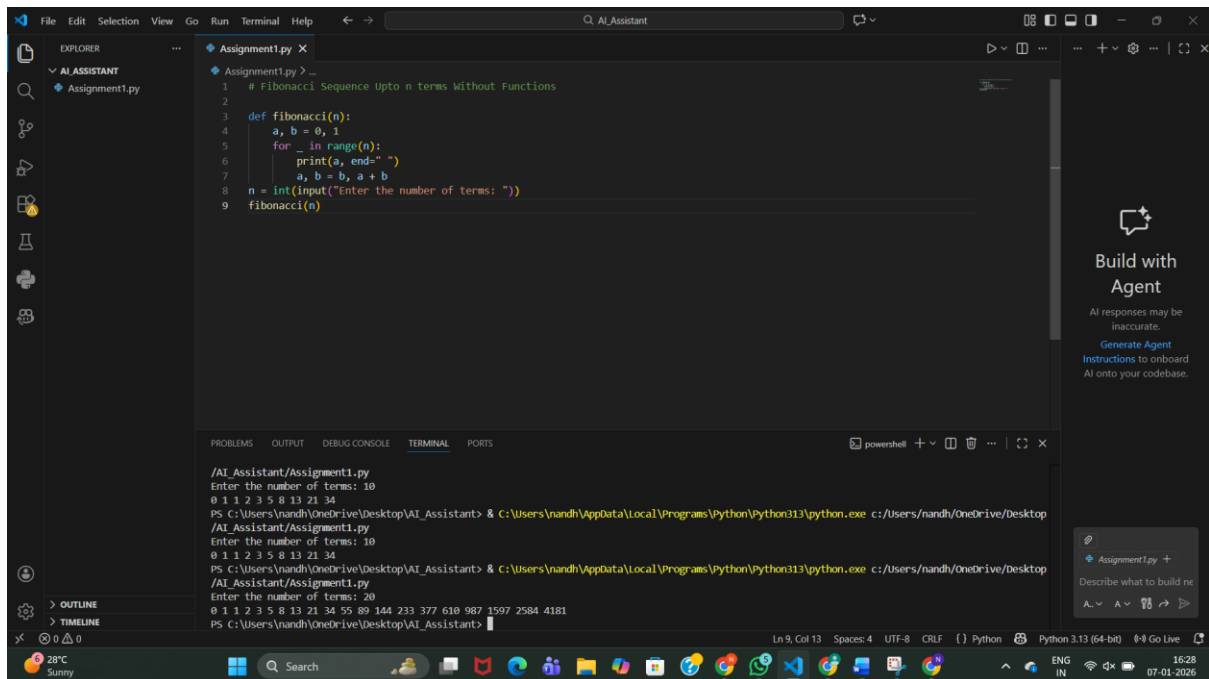
```
n -= 1

def fibonacci(n):
    a, b = 0, 1
    for _ in range(n):
        print(a, end=" ")
        a, b = b, a + b

n = int(input("Enter the number of terms: "))
fibonacci(n)
```

Keep Undo

Result:



Task 4: Comparative Analysis – Procedural vs Modular Fibonacci Code

Comparison: Fibonacci Without Functions vs With Functions

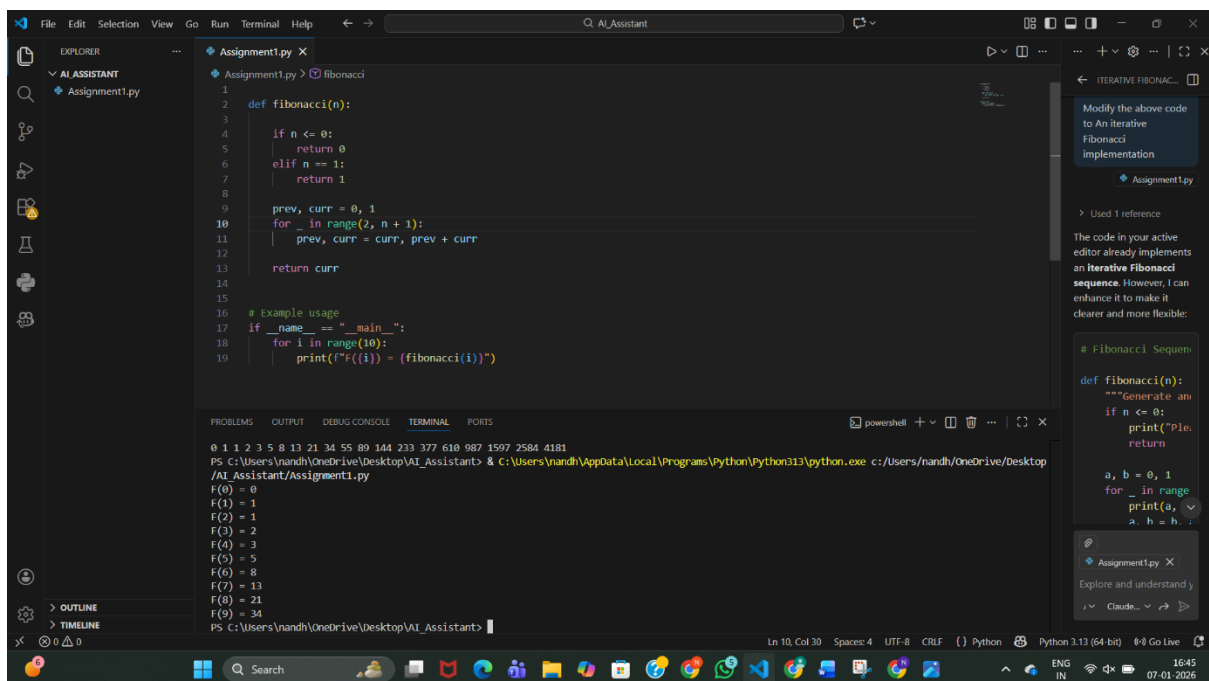
Aspect	Without Functions(Task1)	With Functions (Task 3)
Code Clarity	Logic mixed with input/output	Clear separation of logic
Reusability	Cannot reuse easily	Function can be reused anywhere
Debugging Ease	Harder to isolate errors	Easier to test and debug
Modularity	Not modular	Highly modular
Suitability for Large Systems	Not suitable	Suitable for large applications

Task 5: AI-Generated Iterative vs Recursive Fibonacci Approaches (Different Algorithmic Approaches for Fibonacci Series)

```
Generate code ✓ ✕
Add Context... Claude Haiku 4.5
Keep Undo

2 def fibonacci(n):
3     """Return the nth Fibonacci number."""
4     if n <= 0:
5         return 0
6     elif n == 1:
7         return 1
8
9     prev, curr = 0, 1
10    for _ in range(2, n + 1):
11        prev, curr = curr, prev + curr
12
13    return curr
14
15
16 # Example usage
17 if __name__ == "__main__":
18     for i in range(10):
19         print(f"F({i}) = {fibonacci(i)}")
```

Result:



```
File Edit Selection View Go Run Terminal Help
Assignment1.py X
Assignment1.py > fibonacci
1
2 def fibonacci(n):
3
4     if n <= 0:
5         return 0
6     elif n == 1:
7         return 1
8
9     prev, curr = 0, 1
10    for _ in range(2, n + 1):
11        prev, curr = curr, prev + curr
12
13    return curr
14
15
16 # Example usage
17 if __name__ == "__main__":
18     for i in range(10):
19         print(f"F({i}) = {fibonacci(i)}")

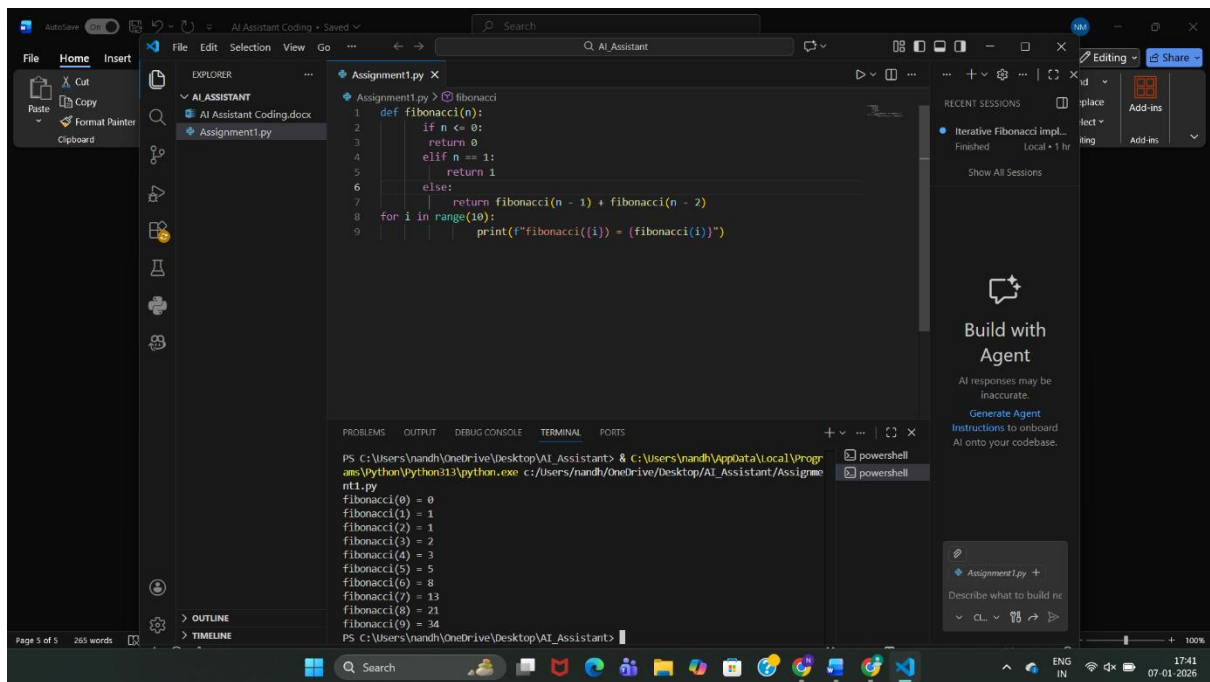
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181
PS C:\Users\nandh\OneDrive\Desktop\AI_Assistant> & C:\Users\nandh\AppData\Local\Programs\Python\Python313\python.exe c:/Users/nandh/OneDrive/Desktop/AI_Assistant/Assignment1.py
F(0) = 0
F(1) = 1
F(2) = 1
F(3) = 2
F(4) = 3
F(5) = 5
F(6) = 8
F(7) = 13
F(8) = 21
F(9) = 34
PS C:\Users\nandh\OneDrive\Desktop\AI_Assistant>
```

Prompt: Generate an recursive Fibonacci implementation

```
Generate code ✓ ✕
Add Context... Claude Haiku 4.5
Keep Undo

def fibonacci(n):
    if n <= 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)
return curr
```

Result:



The screenshot shows a Windows IDE with a dark theme. The main editor window displays a Python file named 'Assignment1.py' containing a recursive Fibonacci function and a loop that prints the first 10 Fibonacci numbers. The code is as follows:

```
1 def fibonacci(n):
2     if n <= 0:
3         return 0
4     elif n == 1:
5         return 1
6     else:
7         return fibonacci(n - 1) + fibonacci(n - 2)
8 for i in range(10):
9     print(f"fibonacci({i}) = (fibonacci(i))")
```

The bottom panel shows the 'TERMINAL' output, which displays the results of the program execution:

```
PS C:\Users\nandh\OneDrive\Desktop\AI_Assistant> & C:\Users\nandh\AppData\Local\Programs\Python\Python313\python.exe c:\Users\nandh\OneDrive\Desktop\AI_Assistant\Assignment1.py
fibonacci(0) = 0
fibonacci(1) = 1
fibonacci(2) = 1
fibonacci(3) = 2
fibonacci(4) = 3
fibonacci(5) = 5
fibonacci(6) = 8
fibonacci(7) = 13
fibonacci(8) = 21
fibonacci(9) = 34
PS C:\Users\nandh\OneDrive\Desktop\AI_Assistant>
```

On the right side of the IDE, there is a sidebar with a 'Build with Agent' section, which includes a warning that 'AI responses may be inaccurate' and a 'Generate Agent' button. Below this, there is a section for 'Assignment1.py' with a 'Describe what to build' button.

Explanation:

Copilot successfully generated both iterative and recursive Fibonacci implementations. The iterative approach was efficient in terms of time and space complexity, while the recursive approach was simpler but computationally expensive for large inputs. This task demonstrated Copilot's understanding of different algorithmic paradigms and their trade-offs.